

MLOps – Session 3: Orchestration, Inference, Serving & Release MLOps Engineering

في هذه السيشن، سننتقل من مجرد بناء النماذج وأتمتة الاختبارات إلى إدارة خطوط الإنتاج المعقدة، واختيار نمط الاستنتاج المناسب (**Inference Patterns**) ، وبناء API خدمة النماذج (**Model Serving**) ، واختبار الأحمال (**Load Testing**) ، واستراتيجيات النشر الآمن (**Release Strategies**) .
تتكون هذه السيشن من 5 مواضيع أساسية:

1. **Orchestration** : استخدام **Apache Airflow** لإدارة الـ DAGs والبنية التحتية والـ Operators .
2. **Inference Patterns** : الفرق بين الـ Web service والـ Batch scoring والـ Streaming ومتى تختار كلاً منها.
3. **Model Serving** : رحلة التدرج من **FastAPI** إلى **BentoML** ثم **Triton / TensorRT** و **ONNX / OpenVINO** وصولاً لخدمة الـ LLMs عبر **vLLM** .
4. **Load Testing** : اختبار الأحمال وقياس الأداء باستخدام **Locust** .
5. **Release Strategies** : استراتيجيات النشر الآمن **Blue/Green** : و **Canary** و **A/B Test** و **Shadow** .

Orchestration (Apache Airflow)

1. لماذا نحتاج إلى منظم المهام (Orchestrator) ؟

في الأنظمة التقليدية، قد يلجأ البعض لإنشاء ملفات سكريبت عشوائية وتحديد مواعيد تشغيلها باستخدام **Cron jobs** . ولكن الـ Cron job ليس خط إنتاج (Pipeline) ؛ فهو يتسبب في مشاكل هندسية خطيرة:

- **بدون Orchestration** : لا يوجد إعادة محاولة تلقائية عند الفشل (**No retry on failure**) ، مما يسبب ضياعاً صامتاً للبيانات (**Silent data loss**) . كما لا توجد رؤية (**No visibility**) لما تم تشغيله أو ما فشل، وتعجز عن تحديد الاعتماديات بين المهام (**Task dependencies**) ، ويتم إعادة تشغيل الكود بالكامل حتى لو لم تتغير البيانات.

- مع وجود **DAG Orchestrator** : يتم تعريف المهام باعتماديات صريحة، مع إعادة المحاولة التلقائية عند الفشل (**Automatic retry with backoff**) ، وتوفر واجهة مستخدم (**Full UI**) لمراقبة الحالة والسجلات والوقت المستغرق لكل مهمة، مع إمكانية إطلاق التشغيل عبر جدول أو مستشعر (**Sensor**) أو حدث (**Event**) ، وتخطي المهام غير المتغيرة (**Incremental runs**) .

2. ما هو الـ DAG؟ (Directed Acyclic Graph)

الـ **DAG** هو الرسم البياني الموجه وغير الدائري الذي يمثل خط الإنتاج:

- **Directed (موجه)** : تسير الأسهم في اتجاه واحد فقط؛ حيث تكون مخرجات المهمة السابقة هي مدخلات المهمة التالية مباشرة.
- **Acyclic (غير دائري)**: لا يحتوي على أي حلقات تكرارية (No loops) ؛ فالمهمة A لا يمكن أن تعتمد على نفسها بشكل مباشر أو غير مباشر.
- مثال لـ **pipeline** إعادة التدريب الأسبوعي:

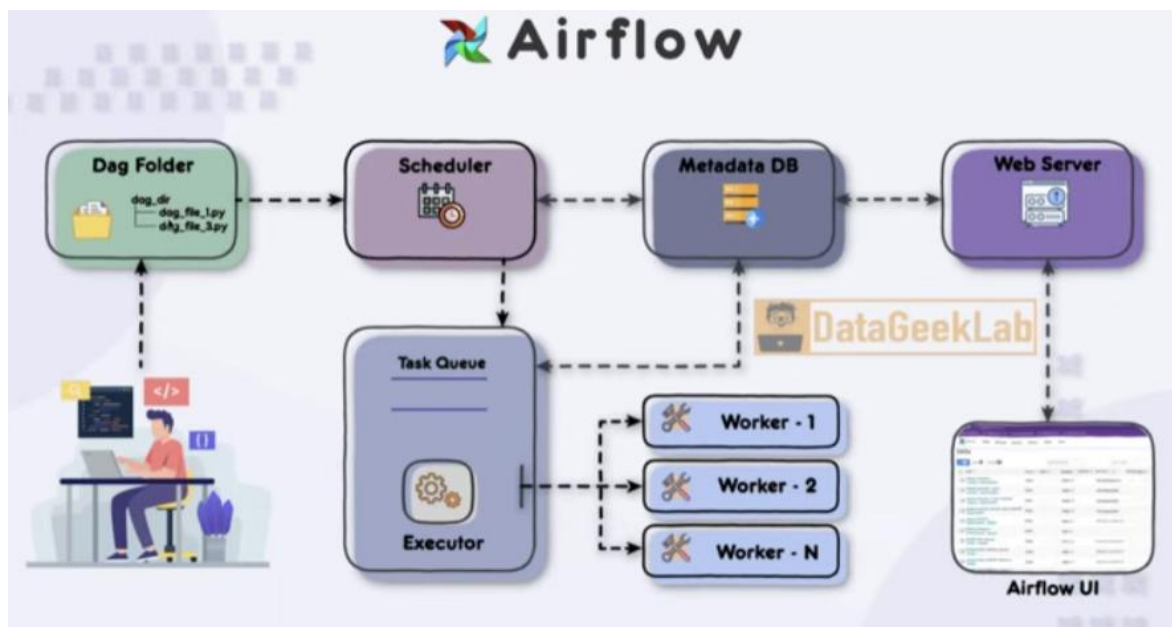
extract_data → validate → train_model → evaluate → register_model

3. البنية التحتية لـ Apache Airflow (5 Components)

يتكون إطار عمل Airflow من خمسة مكونات رئيسية يجب فهمها قبل كتابة أي كود:

1. **Web Server** : واجهة مستخدم تفاعلية مبنية بـ Flask تعمل افتراضياً على البورت 8080 . تتيح لك رؤية مخطط الـ DAG وتطبيق الـ Runs ومراجعة السجلات (Logs) وإعادة تشغيل المهام الفاشلة.
2. **Scheduler** : المحرك الأساسي؛ يقرأ ملفات الـ DAG باستمرار، ويفحص الجداول الزمنية والاعتماديات، ويضع المهام الجاهزة في طابور التنفيذ (**Queue**) .
3. **Executor** : يحدد كيفية تشغيل المهام :
 - **SequentialExecutor** : للتطوير المحلي (ينفذ مهمة واحدة في كل مرة).
 - **LocalExecutor** : لتشغيل المهام بالتوازي على جهاز واحد.
 - **CeleryExecutor** : لتوزيع المهام على عدة أجهزة مختلفة.
 - **KubernetesExecutor** : لتشغيل كل مهمة داخل Pod مستقل في Kubernetes .
4. **Worker** : العملية البرمجية (Process) التي تنفذ كود المهمة فعلياً.

5. **Metadata DB** : قاعدة بيانات (PostgreSQL أو SQLite) تحفظ تعريفات الـ DAGs وحالات المهام والاتصالات والمعاملات؛ وهي المصدر الحقيقي للحقيقة (**Source of truth**) في Airflow .
- كيف تتصل المكونات معاً؟ يقوم الـ **Scheduler** بوضع المهام الجاهزة في الطابور ← ينفذها الـ **Worker** ← تحفظ الـ **Metadata DB** الحالة ← يقرأها الـ **Web Server** ليعرضها في الشاشة.



4. أ. هيكل وبناء الـ DAG (days/retrain_pipeline.py)

يتم تعريف الـ pipeline ككود برمجي داخل مجلد days/ :

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator

dag = DAG(
    dag_id="ride_duration_retrain", # UI والـ CLI اسم فريد يظهر في الـ
    schedule="@weekly", # Cron (أو صيغة) الجدول الزمني
    start_date=datetime(2024, 1, 1), # عدم تشغيل الفترات التاريخية الفاتئة عند البدء
    catchup=False,
    default_args={
        "retries": 2, # عدد محاولات التكرار عند الفشل
        "retry_delay": timedelta(minutes=5), # زمن الانتظار بين المحاولات
        "owner": "ml-team",
    },
)
```

قواعد أساسية عند كتابة كود المهام (Callables) :

- وظائف صغيرة ومستقلة (Idempotent Callables) : اجعل دالة كل مهمة مستقلة وصغيرة لأن Airflow قد يعيد تشغيلها في أي وقت.
- الاستيراد داخل الدالة (Imports inside functions) : استورد المكتبات الثقيلة (مثل mlflow أو pandas) داخل دالة المهمة نفسها وليس في أعلى الملف؛ لأن الـ Scheduler يقرأ ويفحص ملفات الـ DAG باستمرار.

ب. دوال المهام (Callables) داخل الـ DAG

تعريف الدوال الثلاث التي تنفذ خطوات العمل (مع ملاحظة استيراد المكتبات الثقيلة داخل الدوال لحماية الـ Scheduler):

```
def extract(**ctx):
    from google.cloud import storage
    client = storage.Client()
    bucket = client.bucket("mlops-ride-duration")
    bucket.blob("rides/latest.parquet").download_to_filename("data/raw/rides.parquet")

def train(**ctx):
    import mlflow, subprocess
    with mlflow.start_run() as run:
        subprocess.run(["python", "src/train.py"], check=True)
        mlflow.log_metric("train_mae", train_mae)

def evaluate(**ctx):
    best_run = client.search_runs(
        experiment_ids=["1"],
        order_by=["metrics.mae ASC"]
    )
    # Production مع النموذج الحالي في مرحلة MAE مقارنة الـ
```

ربط تسلسل المهام (Task Graph) :

نستخدم الـ operator >> لتحديد ترتيب التنفيذ:

```
t_extract = PythonOperator(task_id="extract", python_callable=extract, dag=dag)
t_train = PythonOperator(task_id="train", python_callable=train, dag=dag)
t_evaluate = PythonOperator(task_id="evaluate", python_callable=evaluate, dag=dag)

# تحديد الاعتماديات: تتبع دالة التدريب عملية التجميع، يليها التقييم
t_extract >> t_train >> t_evaluate
```

قال أنس بن مالك رضي الله عنه عن النبي صلى الله عليه وعلى آله وسلم قال: (يقول الله تعالى لأَهْوَنَ أهل النار عذاباً يوم القيامة: لو أن لك ما في الأرض من شيءٍ أكنّك تفندي به ؟ فيقول نعم. فيقول أردتُ منك أَهْوَنَ من هذا وأنت في صُلب آدم: أن لا تشرك بي شيئاً فأبيت إلا أن تُشرك بي). (حديث صحيح).

5. الـ Operators والـ Sensors

أولاً: الـ Operators (اللبات الأساسية للمهام)

الـ Operator يحدد ماذا تفعل المهمة:

- **PythonOperator** : يطلب وينفذ أي دالة بايثون (الأكثر استخداماً في مشاريع ML).
- **BashOperator** : ينفذ أوامر الشل والـ CLI (مثل تشغيل أمر `dvc repro`).
- **BranchPythonOperator** : يرجع `task_id` للتفرع الشرطي بناءً على النتيجة.
- **EmailOperator** : يرسل تنبيهات عبر البريد عند الانتهاء أو الفشل.

ثانياً: الـ Sensors (مستشعرات شروط الانتظار)

الـ Sensor هو نوع خاص من الـ Operators يظل في حالة انتظار وتوقف حتى يتحقق شرط معين:

- **FileSensor** : ينظر وينتظر حتى يظهر ملف أو مجلد معين في المسار المحلي أو السحابي.
- **ExternalTaskSensor** : ينتظر حتى تكتمل مهمة معينة داخل DAG آخر لمنع التضارب بين أنابيب البيانات المستقلة.
- **HttpSensor** : يقوم بعمل استطلاع (Polling) لـ HTTP endpoint حتى يرجع إجابة بالنجاح (مثل الانتظار حتى يصبح الـ API جاهزاً).

Inference Patterns

في هذا الجزء، سنناقش اختيار النمط المناسب لتشغيل واستنتاج النماذج (Inference Patterns) ، وكيف أن الاختيار الخاطئ يمكن أن يؤدي إلى خسائر مالية فادحة أو فشل كامل للنظام.

1. أغلى خطأ في هندسة الذكاء الاصطناعي (Why Choosing Wrong is Costly)

الخطأ الأكبر في مشاريع الـ ML ليس نشر نموذج بدقة ضعيفة، بل نشر نموذج جيد بالنمط الخاطئ .

هناك ثلاثة سيناريوهات عملية لكوارث اختيار نمط الاستنتاج:

1. انفجار التكلفة (Cost blowout) :

◦ السيناريو: فريق قام بنشر نموذج توصيات (Recommendation Model) كـ REST API

يعمل على سيرفر متصل 7/24.

- النتيجة: تكلفة السيرفر بلغت \$800 شهرياً، رغم أن التوصيات تحتاج للتحديث مرة واحدة ليلاً فقط. لو تم تشغيلها بنظام **Batch Scoring** لكانت التكلفة \$12 شهرياً فقط!
- الأثر: الاختيار الخاطئ جعل النظام أعلى بـ 66 ضعفاً بدون أي داعٍ.

2. اختراق اتفاقية مستوى الخدمة للزمن (Latency SLA breach) :

- السيناريو: فريق قام بنشر نموذج اكتشاف الاحتيال (Fraud Detection) كـ Batch job يعمل كل ليلة.
- النتيجة: العمليات المشبوهة تمر طوال اليوم ويتم اكتشافها في صباح اليوم التالي بعد تسريب 2 مليون دولار في عمليات احتيال.
- الأثر: النموذج أصلح بلا قيمة إطلاقاً حتى لو كانت دقته 99%، لأن نمط الاستنتاج بطيء.

3. المبالغة التعقيدية (Over-engineered) :

- السيناريو: فريق قام ببناء pipeline معقد لـ Data stream باستخدام **Kafka Streaming** لنموذج توقع المبيعات الأسبوعية.
- النتيجة: قضى المطورون 3 أشهر في بناء بنية تحتية للـ Streaming، في حين أن مهمة Batch بسيطة من 10 سطور كود كانت ستؤدي نفس الغرض تماماً.
- الأثر: ضياع 3 أشهر من العمل الهندسي في تعقيد غير مطلوب.

القاعدة الذهبية قبل اختيار أي أداة:

- هل ينتظر المستخدم الإجابة فوراً؟ ← استخدم **Web Service**.
- هل يمكن حساب النتائج مسبقاً؟ ← استخدم **Batch Scoring**.
- هل يجب التفاعل مع الحدث في أقل من ثانية؟ ← استخدم **Streaming**.

عن أبي ذر قال : قال رسول الله صلى الله عليه وعلى آله وسلم: ((يقول الله عز وجل: من جاء بالحسنة فله عشر أمثالها وأزيد، ومن جاء بالسيئة فجزاؤه سيئة مثلها أو أغفر ومن تقرب مني شبرا تقربت منه ذراعاً، ومن تقرب مني ذراعاً تقربت منه باعاً ومن أتاني يمشي أتيته هرولة، ومن لقيني بقراب الأرض خطيئة لا يَشْرِكُ بي شيئاً لقيتُه بمثلها مغفرة)).

(حديث صحيح)

2. مقارنة لأنماط الاستنتاج الثلاثة (The Three Inference Patterns)

مقارنة شاملة بين الأنماط الثلاثة من حيث الاستخدام، الزمن المستغرق، التكلفة، والأدوات:

المعيار	Web Service	Batch Scoring	Streaming
طبيعة النمط	REST API (Request/Response)	Scheduled Job (مهمة مجدولة)	Event-driven (مدفوع بالأحداث)
حالة الاستخدام	عند ضغط المستخدم على "Predict" (مثل: Chatbots, Recommendations, ETA في Uber لـ 100M + طلب/يوم)	حساب نتائج جميع العملاء ليلاً (مثل Spotify : لحساب توصيات 600M مستخدم ليلاً)	التفاعل اللحظي مع كل حدث (مثل : PayPal لتقييم كل عملية دفع في < 200ms)
زمن الاستجابة (Latency)	أقل من 200 مللي ثانية (< 200 ms)	من دقائق إلى ساعات	أقل من ثانية واحدة (< 1 second)
معدل الإنتاجية (Throughput)	1 إلى 1000 طلب/ثانية	ملايين الصفوف لكل عملية	حجم ضخم ومستمر متدفق
التكلفة (Cost)	متوسطة (السيرفر يعمل باستمرار 7/24)	منخفضة جداً (التشغيل أثناء الحساب فقط)	متوسطة (تتوسع الحاويات حسب حجم التدفق)
الأدوات المستعملة	FastAPI / Litestar / BentoML	Python + Parquet + Airflow / Ray	Kafka / Redis Streams + Python

3. النمط الأول : Batch Scoring (معالجة المجموعات المجدولة)

في هذا النمط، لا يوجد مستخدم ينتظر الإجابة فوراً. يتم قراءة البيانات ، حساب التوقعات للجميع، وكتابة النتائج في قاعدة البيانات.

المبادئ البرمجية في كود `src/batch_score.py` :

1. التحميل من الـ **Model Registry** : يتم تحميل النموذج دائماً من مرحلة **Production** في MLflow (`models:/RideDurationModel/Production`) وليس من مسار ثابت. عندما ترقى نموذجاً جديداً لـ **Production** ، يتم سحبه تلقائياً في التشغيل القادم.

2. استخدام **Parquet** بدلاً من **CSV** : صيغة **Parquet** أسرع بـ 10 إلى 100 ضعف من **CSV** لأنها تخزن البيانات عمودياً (Columnar) ، مما يتيح قراءة الأعمدة المطلوبة فقط للتدريب.

3. غياب اتفاقية زمن الاستجابة (No latency SLA) : لا يهم الاستجابة في ثوانٍ، بل التركيز يكون كاملاً على رفع معدل الإنتاجية قليلاً للتكلفة.

4. إضافة عمود تاريخ التشغيل (run_date) : يتم وضع ختم زمني لكل عملية حساب (مثل `df["run_date"] = date.today().isoformat()`) لمعرفة أي batch تسببت في أي نتائج شاذة عند تتبع انحراف البيانات مستقبلاً.

5. الربط مع Airflow : يتم غلق الاسكريبت داخل PythonOperator ليعمل تلقائياً وفق الجدول الزمني المحدد في الـ DAG .

أ. كود الـ Batch Scoring (src/batch_score.py)

يقوم بقراءة ملفات Parquet الضخمة، تحميل النموذج من الـ Registry ، إجراء التوقعات، وضع الختم الزمني، وحفظ النتائج:

```
# src/batch_score.py
import pandas as pd, mlflow.sklearn
from pathlib import Path
from datetime import date

def load_production_model():
    # التحميل دائماً من مرحلة Production في MLflow
    return mlflow.sklearn.load_model("models:/RideDurationModel/Production")

def batch_score(input_path: str, output_path: str) -> None:
    df = pd.read_parquet(input_path)
    print(f"Scoring {len(df):,} rows ...")
    model = load_production_model()
    features = df[["distance_km", "passengers", "hour_of_day"]].values
    df["pred"] = model.predict(features)

    # أنتجت هذه النتائج عند فحص الـ Batch ختم التاريخ لمعرفة أي
    df["run_date"] = date.today().isoformat()
    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    df.to_parquet(output_path, index=False)
    print(f"Done -> {output_path}")

if __name__ == "__main__":
    batch_score(
        input_path = f"data/scoring/input/{date.today()}.parquet",
        output_path = f"data/scoring/output/{date.today()}.parquet",
    )
```

4. النمط الثاني: Streaming Inference (الاستنتاج عبر تدفق الأحداث)

يعمل هذا النمط بشكل مدفوع بالأحداث (Event-driven) لحساب التوقع فور وقوع الحدث مباشرة. المساران الأساسيان للبنية التحتية:

• مسار Apache Kafka : مناسب للإنتاجية العالية جداً، مع إمكانية إعادة تشغيل الأحداث (Replay-able) (المسار :

(AppProducer → KafkaTopic → KafkaConsumer → MLModel → DB

• **مسار Redis Streams** : أسهل في الإعداد ومناسب للتطبيقات ذات زمن التأخير المنخفض والإنتاجية المتوسطة . (المسار

(AppProducer → RedisStream → RedisConsumer → MLModel → DB

ملاحظة: الكود البرمجي للـ (consumer.py) ونموذج الـ ML ثابت ومستقل تماماً ولا يعلم أي Message (broker) يعمل خلفه.

المستودعات الثلاثة للرسائل (Message Brokers) :

1. **Apache Kafka** : القياس العالمي للشركات الكبرى (مثل LinkedIn و Uber) للتعامل مع ملايين الـ Events في الثانية مع حفظ السجلات.
2. **Redis Streams** : ذاكرة مؤقتة منخفضة التأخير وسهلة التركيب مناسبة للأنظمة التي تعالج أقل من 100 ألف حدث/ثانية.
3. **RabbitMQ** : طابور رسائل تقليدي يوصل كل رسالة لمستوعب واحد فقط، أسهل في الفهم للأنظمة البسيطة.

المفاهيم البرمجية لكود consumer.py :

- **الـ Cold Start والتخزين المؤقت (MODEL = None)** : يتم تحميل النموذج مرّة واحدة فقط عند بدء تشغيل الحاوية وتخزينه في الذاكرة العشوائية (RAM)، لتستجيب جميع الـ Events التالية فوراً دون إعادة القراءة من القرص.
- **التوسع الأفقي (Scale out)** : يمكنك تشغيل عدة حاويات من consumer.py بالتوازي، ويقوم Kafka أو Redis بتقسيم الرسائل القادمة بينها تلقائياً لرفع سرعة المعالجة.
- **سياسة إعادة التشغيل في Docker (--restart unless-stopped)** : تضمن أنه في حال تعطل كود الـ Consumer أو إعادة تشغيل السيرفر، يتم إعادة تشغيل الحاوية تلقائياً لضمان عدم توقف الاستقبال.



ب. كود الـ Streaming Consumer (consumer.py)

event handler يعمل باستمرار، يحمل النموذج مرة واحدة عند الـ **Cold Start** ، ويستجيب لكل حدث قادم:

```
# consumer.py - event-driven inference consumer
import json, mlflow.sklearn, os

MODEL = None # التحميل مرة واحدة فقط في الذاكرة لتوفير وقت القراءة من القرص

def get_model():
    global MODEL
    if MODEL is None:
        MODEL = mlflow.sklearn.load_model(os.environ["MODEL_URI"])
    return MODEL

def predict_on_event(event: dict) -> dict:
    """ Broker (Kafka أو Redis) تستدعي مع كل رسالة قائمة من الـ """
    model = get_model()
    payload = json.loads(event["data"])
    features = [payload["distance_km"], payload["passengers"], payload["hour_of_day"]]
    prediction = model.predict([features])

    result = {
        "ride_id": payload["ride_id"],
        "prediction": round(float(prediction), 2)
    }
    store_result(result) # Redis أو PostgreSQL التخزين في
    return result

# python consumer.py مع Docker أو داخل --restart unless-stopped يتم تشغيله كعملية دائمة
```

5. متى تختار Streaming ومقاطعة الـ Batch؟ (Decision Guide)

استخدم الـ Streaming فقط إذا تحقق التالي:

1. القرارات يجب أن تتخذ في أقل من ثانية واحدة بعد وقوع الحدث (مثل اكتشاف الاحتيال والأوامر المالية).
2. تدفق الـ Events مستمر وغير محدود بوقت معين.
3. البيانات تفقد قيمتها التجارية إذا تم تأخيرها لدقائق.
4. كل حدث مستقل بذاته ولا يحتاج لتجميع مع أحداث أخرى.
5. لدى الفريق القدرة التشغيلية لمراقبة وإدارة سيرفرات الـ Consumers المتصلة 7/24.

الترزم بـ Batch Scoring إذا تحقق التالي:

1. النتائج يمكن حسابها مسبقاً يومياً أو ساعياً (مثل حساب النشرات والتوصيات).
2. العملية تتطلب تجميع البيانات عبر أحداث كثيرة (مثل حساب المتوسطات الأسبوعية).
3. عملية استنتاج النموذج بطيئة ($> 500ms$) ، حيث يقلل الـ Batch من تكلفة التشغيل.
4. لم تكن متأكداً بعد؛ ابدأ دائماً بـ Batch Scoring واسترخ، ثم انتقل لـ Streaming فقط عند وجود حاجة فعلية مقاسة!

Model Serving

نحن جابونا علي سؤال: "ما هو نمط الـ **Inference** المناسب؟"، والأن سنجيب على سؤال: "كيف تبني نظام خدمة نماذج (Serving System) يعمل بكفاءة واستقرار تحسبي تحت الضغط المالي والهندسي الحقيقي؟"

1. ما هو الـ Model Serving ؟ وفجوة الخدمة (The Serving Gap)

أولاً: تعريف الـ Model Serving

هو عملية تحويل نموذج التعلم الآلي المدرب إلى واجهة برمجية (API) أو خدمة يمكن توزيعها واستهلاكها بسهولة بواسطة التطبيقات، مع استضافتها على بنية تحتية قادرة على التوسع والتعامل مع الأحمال العالية.

SERVING = INFERENCE + EVERYTHING AROUND IT

- **Inference (الاستنتاج)** : هو مجرد إجراء عملية رياضية واحدة ($model(x)$) لحساب المدخلات والمخرجات (Forward pass).
- **Serving (الخدمة)**: هو البناء الهندسي المتكامل المحيط بتلك العملية الرياضية لتوفير الـ HTTP/gRPC endpoints ، وإدارة الاعتمادية، والتحكم في تزامن الطلبات، والتوسع، وإمكانية المراقبة والتحديث دون تعطيل الخدمة.

ثانياً: فجوة الخدمة (The Serving Gap)

- تحدث الكارثة المعتادة في معظم المشاريع عندما يستغرق الفريق شهوراً لتدريب نموذج وتصل دقته لـ 94%، ثم يطرح قسم المنتج السؤال الحاسم: "أين نرسل البيانات للحصول على التوقع؟" فتصمت الغرفة!
- **مرحلة التدريب (Training)** : مسألة بحثية (Research Problem) تركز على الدقة، الهيكلية، وجودة البيانات.
 - **مرحلة الخدمة (Serving)** : مسألة أنظمة (Systems Problem) تركز على زمن التأخير (Latency)، معدل الإنتاجية (Throughput)، التزامن (Concurrency)، والتكلفة.
 - **القاعدة الأساسية**: ملف النموذج ($model.pth$ أو $model.pkl$) ليس منتجاً، بل هو مجرد ملف على القرص حتى يتحول إلى رابط URL يستجيب في أقل من 40 مللي ثانية تحت حمل 500 طلب/ثانية.

عن أبي هريرة عن النبي صلى الله عليه وعلى آله وسلم قال: ((قال الله عز وجل: سَبَقْتُ رَحْمَتِي غَضَبِي))

أستغفر الله العظيم الذي لا إله إلا هو الحي القيوم وأتوب إليه

2. قاموس مصطلحات الخدمة الحرج (The Serving Vocabulary)

هناك 6 مصطلحات أساسية تحكم كل قرار هندسي في هذا المجال:

1. **Latency (زمن الاستجابة):** الوقت المستغرق من لحظة إرسال الطلب حتى استقبال الإجابة (وهو ما يشعر به المستخدم).
2. **p50 / p95 / p99 (النسب المئوية لزمن الاستجابة):** المتوسط الحسابي مضلل جداً! فـ **p99** يمثل أبداً 1% من الطلبات، وهو الرقم الفعلي الذي يتسبب في كسر اتفاقية مستوى الخدمة (SLA).
3. **Throughput (معدل الإنتاجية):** عدد الطلبات التي تمت خدمتها في الثانية الواحدة (وهو المحدد الرئيسي لفاتورة البنية التحتية).
4. **Batch size (حجم الدفعة):** عدد المدخلات المعالجة في خطوة inference واحدة، وهو المحرك الرئيسي للتوازن بين الـ Latency والـ Throughput.
5. **GPU utilization (نسبة استغلال الـ GPU):** نسبة الطاقة الحسابية الفعلية التي تعمل داخل كارت الشاشة.
 - **حقائق صادمة:** النماذج التي تنشر عبر FastAPI تقليدياً تستغل فقط 8% إلى 15% من قدرة الـ GPU! وهذه الفجوة هي السبب الرئيسي لوجود أدوات الـ Serving المتقدمة.
6. **Cold start (البداية الباردة):** الوقت المستغرق من لحظة تشغيل الحاوية حتى جاهزية النموذج لتلقي أول طلب (وهو ما يقتل التصميم المعتمد على الـ Serverless و Autoscaling).

3. الطبقات الأربع لمنظومة الخدمة (The 4-Layer Serving Stack)

أي أداة خدمة نستخدمها تسيطر على جزء أو كل هذه الطبقات الأربع:

```
[ Layer 1: Transport & API Layer ] ← HTTP/gRPC, Auth, Pydantic Validation, OpenAPI docs
[ Layer 2: Service & Scheduling ] ← Dynamic batching, Request queueing, Model versioning, Concurrency
[ Layer 3: Inference Runtime ] ← PyTorch eager / ONNX Runtime / TensorRT / OpenVINO
[ Layer 4: Hardware Infrastructure ] ← NVIDIA GPU (A100/H100), Intel CPU (AVX-512), Edge
```

ملكية الأدوات لهذه الطبقات (Tool Ownership):

- **FastAPI** : يمتلك Layer 1 فقط.
- **BentoML** : يمتلك Layer 1 + Layer 2
- **Triton** : يمتلك Layer 1 + Layer 2 + Layer 3
- **vLLM** : يمتلك Layer 1 + Layer 2 + Layer 3 (مخصص بالكامل للـ LLMs).

4. التدرج الهندسي للخدمة (The Serving Progression: 5 Categories)

ينقسم اختيار أدوات الخدمة إلى 5 فئات متدرجة، حيث لا ننتقل للفئة التالية إلا إذا تم قياس الحاجة إليها عبر اختبار الأحمال:

```
[CAT 1: FastAPI] —(GPU <15%?)—> [CAT 2: BentoML] —(Need GPU accel?)—> [CAT 3: Triton+TRT]
                                     |
                                     (No GPU?) —> [CAT 4: ONNX/OpenVINO]
                                     |
                                     (Is LLM?) —> [CAT 5: vLLM]
```

الفئة الأولى (Native Python (FastAPI / Flask)) :

- المواصفات : GPU Utilization: 8–15% | Throughput ~10 req/s
 - الاستخدام: النماذج الأولية (Prototypes) ، والأدوات الداخلية التي تستقبل أقل من 50 طلب/ثانية.
 - مشاكل FastAPI الأربعة عند الأحمال العالية :
1. **Batch size = 1** دائماً: الكارت مصمم لمعالجة 32–64 صورة معاً، بينما يرسل FastAPI صورة واحدة لكل طلب.
 2. **Python GIL blocks concurrency** : تشغيل model(img) يوقف الـ Event loop في بايثون ويجمد بقية الطلبات.
 3. **Eager PyTorch is slow** : تنفيذ العمليات بطيء دون تحسين للرسم البياني (No graph fusion).
 4. غياب إدارة الإصدارات والـ **Health checks** : لا يوفر Tracking points تلقائية للنظام.

أ. المستوى الأول: تطبيق FastAPI الأساسي (main.py)

```
from fastapi import FastAPI, UploadFile
import torchvision, torch, io
from PIL import Image

app = FastAPI()
model = torchvision.models.resnet50(weights="DEFAULT").eval().cuda()

transform = torchvision.transforms.Compose([
    torchvision.transforms.Resize(256),
    torchvision.transforms.CenterCrop(224),
    torchvision.transforms.ToTensor(),
])

@app.post("/predict")
async def predict(file: UploadFile):
    img = transform(Image.open(io.BytesIO(await file.read()))).unsqueeze(0).cuda()
    with torch.inference_mode():
        logits = model(img)
    return {"class_id": int(logits.argmax()), "score": float(logits.max())}

# uvicorn main:app --host 0.0.0.0 --port 8000
```

الفئة الثانية: ML Framework (BentoML)

- المواصفات: Throughput ~200 req/s | GPU Utilization: 40–55%.
- الاستخدام: النقطة الافتراضية للبدء لمعظم مشاريع الإنتاج الحقيقية (حل 90% من المشاكل).
- كيف يحل BentoML مشاكل FastAPI؟
 - Micro-batching**: تفعيل خيار `batchable=True` يجمع الطلبات المتزامنة تلقائياً في دفعة واحدة ويرسلها للنموذج.
 - الـ Runner كمتوّر مستقل**: يعزل النموذج في عملية برمجية منفصلة (**Separate Process**) لمنع انسداد الـ GIL وتظل واجهة الـ HTTP سريعة وغير متوقفة.
 - إدارة الإصدارات: وسم النماذج تلقائياً عبر `bentoml.sklearn.save_model()`.
 - نقاط فحص حية جاهزة: يتيح نقاط فحص صحة النظام تلقائياً عبر `/healthz` و `/readyz`.

ب. المستوى الثاني: خدمة BentoML مع الـ Micro-batching (service.py)

```
import bentoml, numpy as np, mlflow.sklearn
from pydantic import BaseModel

# 1. (micro-batching) وتمكين النجم المجهري BentoML حفظ النموذج في مخزن
sk_model = mlflow.sklearn.load_model("models:/RideDurationModel/Production")
bento_model = bentoml.sklearn.save_model(
    "ride_duration", sk_model,
    signatures={"predict": {"batchable": True, "batch_dim": 0}},
)

# 2. GIL في عملية برمجية منفصلة لمنع الـ Runner والـ Service تعريف الـ
class PredictRequest(BaseModel):
    distance_km: float; passengers: int; hour_of_day: int

class PredictResponse(BaseModel):
    duration_min: float

runner = bentoml.sklearn.get("ride_duration:latest").to_runner()
svc = bentoml.Service("ride_api", runners=[runner])

@svc.api(input=bentoml.io.JSON(pydantic_model=PredictRequest),
         output=bentoml.io.JSON(pydantic_model=PredictResponse))
async def predict(req: PredictRequest) -> PredictResponse:
    features = np.array([[req.distance_km, req.passengers, req.hour_of_day]])
    result = await runner.predict.async_run(features) # استدعاء غير حائل
    return PredictResponse(duration_min=float(result))
```

الفئة الثالثة: GPU-Accelerated (TensorRT + NVIDIA Triton)

- المواصفات : GPU Utilization: 80–92% | Throughput ~2000 req/s
- متى ننتقل لها؟ عندما تضغط تكلفة الـ GPU على الميزانية وتصبح الـ p99 latency عائقاً حاسماً.
- الروافع الثلاثة لتسريع الـ GPU :

1. **Graph Optimization** : تجميع كود الرسم البياني لدمج العمليات (مثل دمج Conv+BN+ReLU في Kernel واحد) عبر **TensorRT** أو **ONNX**.

2. **Reduced Precision** : تقليل الدقة الحسابية من FP32 إلى FP16 أو INT8 (معايرة INT8 تضمن انخفاضاً أقل من 0.5% في الدقة مقابل تضاعف السرعة).

3. **Device Concurrency** : تشغيل عدة نسخ من النموذج على نفس كارت الـ GPU عبر `instance_group` لمنع توقف الكارت أثناء نقل البيانات من وإلى الذاكرة.

التكامل بين **BentoML** و **Triton** :

لا توجد منافسة بينهما؛ **BentoML** يمتلك طبقة الـ HTTP والـ Logic والتحقق (Layer 1) ، بينما يرسل المتجهات إلى **Triton** ليقود محرك الـ GPU والـ **TensorRT (Layers 2+3)**.

1. تصدير النموذج إلى صيغة **ONNX** :

```
# export_onnx.py
import torch, torchvision

model = torchvision.models.resnet50(weights="DEFAULT").eval().cuda()
dummy = torch.randn(1, 3, 224, 224, device="cuda")

torch.onnx.export(
    model, dummy, "model.onnx",
    input_names=["input"], output_names=["output"],
    dynamic_axes={"input": {0: "batch"}, "output": {0: "batch"}} # تمكين البُعد المتغير للدقة
)
```

2. ملف إعدادات **Triton** الخالي من الأكواد (`config.pbtxt`) :

```
name: "resnet50"
backend: "onnxruntime" # أو "tensorrt"
max_batch_size: 64

dynamic_batching {
    preferred_batch_size:
        max_queue_delay_microseconds: 5000 # انتظار حتى 5 مللي ثانية لتجميع الدفعة
}

instance_group [{ count: 2, kind: KIND_GPU }] # لنوازي المعالجة GPU تشغيل نسختين على كارت الـ
```

3. ربط BentoML مع سيرفر Triton (service_triton.py) :

```
import bentoml, numpy as np
import tritonclient.http as httpclient

class TritonRunner(bentoml.Runner):
    def __init__(self):
        self.client = httpclient.InferenceServerClient("localhost:8000")
        super().__init__(name="triton_resnet50")

    def run(self, inputs: np.ndarray) -> np.ndarray:
        inp = httpclient.InferInput("input", inputs.shape, "FP32")
        inp.set_data_from_numpy(inputs)
        result = self.client.infer("resnet50", [inp])
        return result.as_numpy("output")

runner = TritonRunner()
svc = bentoml.Service("resnet_prod", runners=[runner])
```

الفئة الرابعة : CPU-Accelerated (ONNX Runtime / OpenVINO)

(التسريع على الـ CPU بـ ONNX Runtime و OpenVINO)

- المواصفات : Throughput ~500 req/s على الـ CPU بدلاً من الـ GPU.
 - متى ننتقل لها؟ في عدم وجود GPU ، أو عند تحسين تكلفة السحابة (خوادم الـ CPU أرخص بـ 10 أضعاف) ، أو عند النشر على الأجهزة الطرفية (Edge Deployment).
 - الخيارات المتاحة :
- ONNX Runtime (Static INT8) : يعمل على أي بيئة برمجية (x86 أو ARM) وأسرع بـ 3 إلى 6 أضعاف من PyTorch الكلاسيكي على الـ CPU.
 - Intel OpenVINO : مخصص لمعالجات Intel وتعليمات AVX-512 الحسابية، ويعطي أعلى أداء ممكن على الـ CPU (أسرع بـ 5 إلى 8 أضعاف).

1. التحويل لـ INT8 باستخدام ONNX Runtime (cpu_serving.py) :

```
from onnxruntime.quantization import quantize_static, CalibrationDataReader, QuantType
import onnxruntime as ort, numpy as np

class MyCalibReader(CalibrationDataReader):
    def __init__(self, X):
        self.data = iter([{"input": X[i:i+8].numpy()} for i in range(0, len(X), 8)])
    def get_next(self): return next(self.data, None)

quantize_static(
    "model.onnx", "model_int8.onnx",
    CalibrationDataReader=MyCalibReader(X_calib),
    quant_format=QuantType.QInt8
)

# التشغيل على الـ CPU
sess = ort.InferenceSession("model_int8.onnx", providers=["CPUExecutionProvider"])
out = sess.run(None, {"input": X.numpy()})
```

2. التشغيل المسرع على معالجات Intel بـ OpenVINO (openvino_serving.py) :

```
from openvino.runtime import Core
from openvino.tools.mo import convert_model

ov_model = convert_model("model.onnx")
core = Core()
compiled = core.compile_model(ov_model, "CPU")
infer_req = compiled.create_infer_request()

output = infer_req.infer({"input": X.numpy()}["output"])
```

الفئة الخامسة: LLM Serving (vLLM / SGLang / TGI)

• المواصفات : GPU Utilization: 85–95% | TTFT ~200ms .

• لماذا تحتاج نماذج اللغات الكبيرة (LLMs) إلى منظومة مختلفة تماماً؟

◦ **Compute shape** : التوليد التكيفي المتسلسل (**Autoregressive**) يتطلب N من التمريرات لكل توكن جديد بدلاً من تمريرة واحدة.

◦ **KV Cache fragmentation** : ذاكرة الـ KV Cache تنمو مع طول النص الممرر وتتسبب في إهدار حتى 80% من ذاكرة الـ GPU .

بدون PagedAttention ← ذاكرة GPU مقسمة يدوياً ومدروسة كتل ثابتة (إهدار 80% من الذاكرة)
باستخدام PagedAttention ← ذاكرة غير متصلة تقسم مثل الـ Virtual Memory في أنظمة التشغيل
(تضاعف الإنتاجية 3-24 ضعف)

مميزات محرك vLLM الأساسية:

1. **PagedAttention** : إدارة ذاكرة الـ KV Cache بكفاءة لمنع التشتت.

2. **Continuous Batching** : إدخال الطلبات الجديدة في منتصف عملية التوليد للطلبات الحالية دون انتظار انتهاء الدفعة.

3. **OpenAI-Compatible API** : تشغيل السيرفر بأمر واحد عبر `vllm serve <model>` ليصبح بديلاً متوافقاً تماماً مع مكتبة OpenAI البرمجية دون تعديل أكوادك.

4. Streaming Tokens : إرجاع التوكنز للمستخدم لحظة توليدها عبر SSE streaming ليكون زمن التفاعل لـ TTFT ممتازاً جداً.

```
# 1. عبر الـ CLI تشغيل السيرفر المتوافق مع 1.
# vllm serve Qwen/Qwen2.5-7B-Instruct --host 0.0.0.0 --port 8000 --max-model-len 8192

# 2. استدعاء السيرفر باستخدام OpenAI SDK
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1", api_key="EMPTY")

# استدعاء عادي
response = client.chat.completions.create(
    model="Qwen/Qwen2.5-7B-Instruct",
    messages=[{"role": "user", "content": "Explain PagedAttention."}],
    max_tokens=256, temperature=0.7,
)

# 3. استدعاء تدفق التوكنز (Streaming Tokens)
stream = client.chat.completions.create(
    model="Qwen/Qwen2.5-7B-Instruct",
    messages=[{"role": "user", "content": "Write a haiku about MLOps."}],
    stream=True,
)
for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="", flush=True)

# 4. HTTP المباشر بدون Python In-Process API استخدام الـ
from vllm import LLM, SamplingParams

llm = LLM(model="Qwen/Qwen2.5-7B-Instruct", dtype="float16")
params = SamplingParams(temperature=0.8, max_tokens=512)
out = llm.generate(["Tell me about MLOps."], params)
```

قال عبد الله بن عمرو بن العاص يقول: قال رسول الله صلى الله عليه وعلى آله وسلم: ((إن الله سيُخَلِّصُ رجلاً من أمتي على رءوس الخلائق يوم القيامة فيُنشَرُ عليه تسعة وتسعين سجلاً كلُّ سَجَلٍ مثل مدِّ البصر ثم يقول: أُنْكِرُ من هذا شيئاً؟ أَظْلَمَكَ كَتَبَتِي الحافظون ؟ فيقول: لا يارب فيقول: أَفْلَكَ عُذْرٌ؟ فيقول: لا يارب فيقولُ بلى إن لك عندنا حسنةً فإنه لا ظلم عليك اليوم فتخرجُ بطاقةً فيها أشهدُ أن لا إله إلا الله وأشهدُ أن محمداً عبده ورسوله فيقول: احضر وَزَنَكَ فيقول: يارب ما هذه البطاقةُ مع هذه السجلات فقال إنك لا تُظلم قال: فتوضع السجلاتُ في كفةٍ والبطاقةُ في كفةٍ فطاشتِ السجلاتُ وثقلتِ البطاقةُ فلا يَنقُلُ مع اسم الله شيءاً)). (حديث صحيح)

5. مقارنة الاستضافة Managed Cloud vs Self-Hosted :

سنستعرض خيارات الاستضافة السحابية المدارة مقابل الاستضافة الذاتية:

Managed Cloud (AWS SageMaker / GCP Vertex)	Self-Hosted (BentoML / Triton)	المعيار
دقائق معدودة عبر الشاشات المباشرة.	من ساعات إلى أيام.	زمن الإعداد
مرتفعة (بسبب الحدود الأدنى لتشغيل الخدمة).	منخفضة (دفع قيمة السيرفر الفعلي فقط).	التكلفة في الأحمال المنخفضة
مرتفعة جداً (رسوم إضافية لكل عملية توقع).	منخفضة ومتحكم بها.	التكلفة في الأحمال العالية
محدود بحسب المحركات التي يتيحها المزود.	تحكم كامل في الـ Quantization والـ Engine.	التحكم بالـ GPU
تقيد بأكواد وواجهات المزود السحابي.	تحكم كامل وحاويات مرنة دون التقيد بمزود واحد.	الخصوصية والـ Vendor Lock-in

Load Testing

1. ما هو اختبار الأحمال؟ ولماذا تحتاجه نماذج الـ ML تحديداً؟

اختبار الأحمال هو محاكاة هجوم متزامن لمئات أو آلاف المستخدمين على واجهة الـ API الخاصة بك لملاحظة كيفية تصرف النظام تحت الضغط الواقعي.

لماذا تختلف خدمات الـ ML عن واجهات الـ Web التقليدية؟

- **بطء الاستنتاج**: عملية التنبؤ الرياضي (`predict()`) أبطأ بكثير من مجرد الاستعلام من قاعدة بيانات كلاسيكية.
- **عائق الـ Python GIL**: يتسبب التنافس على الـ (Threads) داخل بايثون في حدوث Performance Cliff وتجميد الطلبات.
- **البداية الباردة (Cold Start)**: تحميل أوزان النموذج عند بدء الحاوية يضيف قفزات غير متوقعة في زمن التأخير.
- **تراكم الذاكرة (Memory Leaks)**: تنمو الذاكرة العشوائية بصمت حتى يسقط السيرفر بخلل استهلاك الذاكرة (OOM Crash) عند وصول عدد المستخدمين لـ N.

2. مقاييس اختبار الأحمال في أنظمة الـ LLMops

عند التعامل مع نماذج اللغات الكبيرة (LLMs) ، تتغير أبعاد القياس لتشمل أربعة مقاييس رئيسية:

1. **TTFT (Time to First Token)** : الوقت المستغرق حتى ظهور أول توكن للمستخدم (وهو المقياس التجريبي الأساسي لتجربة المستخدم).
2. **TBT (Time Between Tokens)** : السرعة بين التوكنز المتتالية؛ ويجب أن تكون أقل من 50 مللي ثانية لكل توكن لضمان تدفق سلس.
3. **Concurrency (التزامن)** : يوضح الاختبار مدى كفاءة الـ Continuous Batching داخل vLLM لتحديد حجم الدفعة الأنسب للـ SLA .
4. **Context Length (طول السياق)**: النصوص الطويلة تستهلك ذاكرة أكبر؛ مما يفرض إجراء الاختبار عند طول السياق المعتاد لـ p95 .
5. **سكريبت محاكاة الأحمال والتحقق من الاستجابة (locustfile.py)**

```
# locustfile.py
from locust import HttpUser, task, between
import random

class MLAPIUser(HttpUser):
    wait_time = between(0.5, 2.0) # محاكاة انتظار المستخدم بين 0.5 و 2 ثانية

    @task(weight=10) # health أضعاف احتمالية الاستدعاء مقارنة بـ 10
    def predict(self):
        payload = {
            "distance_km": round(random.uniform(0.5, 30.0), 2),
            "passengers": random.randint(1, 4),
            "hour_of_day": random.randint(0, 23),
        }
        with self.client.post("/predict", json=payload, catch_response=True) as resp:
            if resp.status_code != 201:
                resp.failure(f"Expected 201, got {resp.status_code}")
            elif resp.json().get("duration_min", -1) < 0:
                resp.failure("Negative duration generated") # فحص منطقي للنتائج

    @task(weight=1)
    def health(self):
        self.client.get("/health")

# CI/CD: بدون واجهة Terminal أمر التشغيل المباشر عبر الـ
# locust -f locustfile.py --host http://localhost:8000 --users 100 --spawn-rate 10 --
run-time 2m --headless --csv=results/load
```

- **catch_response=True** : تتيح لك توفير Tag الاستجابة ك Failure يدوياً حتى لو كان رمز الاستجابة 200 OK (مثلاً إذا أرجعت النماذج نتائج غير منطقية أو قيم سالبة).
- **التشغيل التلقائي (--headless)** : تشغيل Locust عبر الـ CLI دون واجهة الويب واستخراج نتائج stats.csv لإلغاء الـ Build في الـ CI/CD إذا تجاوزت الـ p95 الحد المسموح.

3. مقارنة أدوات اختبار الأحمال (Tools Comparison)

يقارن المنهج بين خمس أدوات رئيسية لاختبار الأحمال:

نقاط القوة	الأداة	البيئة	الاستخدام الأنسب
سيناريوهات بايثون خالصة، واجهة ويب، تشغيل Headless/CSV للـ CI/CD .	Locust (الموصى بها)	Python	فرق الـ ML والـ LLMOps (نفس لغة الموديل).
أداء سريع جداً بفضل محرك Go ، تقارير Grafana سحابية.	k6	JS / Go	فرق الـ Frontend والـ Full-stack
إعدادات YAML بدون أكواد معقدة، تدعم AWS Lambda	Artillery	Node.js	اختبارات HTTP سريعة.
تقارير تفاعلية ممتازة للأنظمة الضخمة.	Gatling	JVM / Scala	بيئات الشركات ذات المقياس العالي.
أداة تقليدية قديمة تدعم بروتوكولات متعددة.	Apache JMeter	Java GUI	فرق الـ QA والأنظمة القديمة.

Release Strategies (استراتيجيات النشر الآمن)

1. المشكلة التي نحلها

لديك نموذج جديد v2 حقق تحسناً بنسبة 12% في دقة MAE . إذا قمت بالنهج البدائي (إيقاف السيرفر، استبدال النموذج، وإعادة التشغيل)، فإنك تتسبب في انقطاع الخدمة (Downtime) ، وإذا يحتوي الموديل على خلل غير متوقع سيتأثر جميع المستخدمين فوراً.

الانتقال يتم عبر أربعة استراتيجيات رئيسية لتحديث الخدمة بأمان:

2. الاستراتيجيات الأربعة بالتفصيل

1. Blue / Green Deployment (التبديل اللحظي)

- **المفهوم:** تشغيل بيئتين متطابقتين تماماً (Blue تحتوي على الموديل الحالي، و Green تحتوي على الموديل الجديد). يتم توجيه 100% من المرور إلى Blue ، وعند التأكد من جاهزية Green ، يتم تبديل موجه المرور (Load Balancer) لحظياً.
- **المميزات:** انقطاع صفر للمستخدمين (Zero downtime) ، وإمكانية تراجع لحظية (Instant Rollback) بالعودة لـ Blue .
- **العيوب :** مضاعفة تكلفة البنية التحتية (2x Compute) أثناء تشغيل البيئتين معاً.

2. Canary Release (الإطلاق التدريجي)

- **المفهوم:** توجيه نسبة ضئيلة جداً من المرور (مثلاً 5%) للنموذج الجديد ومراقبة المقاييس برهة من الزمن. إذا استقرت النتائج، ترتفع النسبة تدريجياً: 5% → 20% → 50% → 100% .
- **المميزات:** حصر الأثر عند حدوث أخطاء لنسبة ضئيلة من المستخدمين، والتراجع السريع عند استشعار تدهور المقاييس.
- **الاستخدام:** الأسلوب القياسي والافتراضي المفضل لغالبية نماذج الماشين ليرنينج.

3. A/B Testing (الاختبار الموجه للعمل)

- **المفهوم :** تقسيم المستخدمين بشكل محدد وثابت (Deterministic Segmentation) . المجموعة A تستخدم الموديل الأول والمجموعة B تستخدم الموديل الثاني.
- **الهدف:** قياس أثر الأعمال التجاري (معدل النقر CTR ، المبيعات، ومعدل الاحتفاظ) وليس المقاييس التقنية فقط كـ MAE .

4. Shadow Mode (الوضع الخفي صفر الخطورة)

- **المفهوم :** إرسال نسخة من كل طلب حقيقي متوازي للموديل الجديد واستخراج النتيجة لتسجيلها فقط دون إرجاعها للمستخدم.
- **المميزات :** خطر صفر تماماً على تجربة المستخدم مع إمكانية التقييم على بيانات حية.
- **الاستخدام:** النماذج حرجة للغاية والتغييرات الهيكلية الكبيرة.

3. الجدول المقارن للاستراتيجيات الأربعة

هذا ملخص لتحديد الاختيار المناسب:

الاستراتيجية	تكلفة البنية التحتية	خطر المستخدم	التراجع (Rollback)	قياس الأثر التجاري	الاستخدام الأنسب
Blue / Green	2x أثناء التشغيل	صفر أثناء التبديل	لحظي (Flip LB)	لا (فحص تقني)	متطلبات SLA صارمة للتراجع.
Canary	+1x ~نسبة ضئيلة	أقل من 5% من الحركة	سريع (Re-weight)	جزئي (الأخطاء الحية)	الخيار الافتراضي لغالبية التحديثات.
A/B Test	2x ~ (موديلين مباشرين)	50% من المستخدمين	بطيء (تحتاج دلالة إحصائية)	نعم (تحويل ومبيعات)	قياس التأثير على أهداف العمل.
Shadow Mode	2x ~ (استنتاج مضاعف)	صفر	N/A (لا أثر)	لا (تقييم غير مباشر)	التغييرات عالية الخطورة.

أ. إعدادات Blue/Green و Canary عبر Nginx (docker-compose.yml & nginx.conf)

1. ملف docker-compose.yml للبيئة:

```
services:
  nginx:
    image: nginx:alpine
    ports: ["8000:80"]
    volumes: [".:/nginx.conf:/etc/nginx/nginx.conf"]
    depends_on: [blue, green]

  blue: # النسخة الحالية في الإنتاج
    image: yourusername/ride-api:v1.2.0

  green: # النسخة الجديدة المرشحة
    image: yourusername/ride-api:v1.3.0
```

2. ملف `nginx.conf` لتوزيع الوزن في الـ `Canary` (مثلاً 95% لـ `Blue` و 5% لـ `Green`):

```
upstream backend {
    server blue:8000 weight=95;    # للموديل الحالي 95%
    server green:8000 weight=5;    # للموديل الجديد 5%
}

server {
    listen 80;
    location / {
        proxy_pass http://backend;
    }
}

# التحديث اللحظي بدون إسقاط اتصالات: nginx -s reload
```

عند تحديث الأوزان في Nginx ، نقوم فقط بتشغيل الأمر `nginx -s reload` لإعادة تحميل الإعدادات لحظياً ودون إسقاط أي اتصال حائل.

ب. كود تقسيم المستخدمين الثابت في الـ `A/B Testing` (`ab_test.py`)

```
# ab_test.py
import hashlib
from fastapi import Request, FastAPI

app = FastAPI()

def get_model_for_user(user_id: str):
    # توجيه نفس المستخدم لنفس الموديل دائماً باستخدام MD5 hash
    bucket = int(hashlib.md5(user_id.encode()).hexdigest(), 16) % 100
    return model_a if bucket < 50 else model_b

@app.post("/predict")
async def predict(req: PredictRequest, request: Request):
    user_id = request.headers.get("X-User-ID", "anonymous")
    model = get_model_for_user(user_id)
    result = model.predict([[req.distance_km, req.passengers, req.hour_of_day]])

    log_event(
        user_id=user_id,
        variant="A" if model == model_a else "B",
        prediction=result
    )
    return {"duration_min": round(float(result), 2)}
```

ج. كود الـ Shadow Mode الخفي غير الحاجز (shadow.py)

```
# shadow.py
import asyncio
from fastapi import FastAPI

app = FastAPI()

@app.post("/predict")
async def predict(req: PredictRequest):
    features = [[req.distance_km, req.passengers, req.hour_of_day]]

    # 1. إرجاع النتيجة من الموديل الرئيسي للمستخدم فوراً
    prod_result = production_model.predict(features)

    # 2. تقييم الموديل الخفي في الخلفية دون تعطيل وقت الاستجابة للمستخدم
    async def shadow_eval():
        shadow_result = shadow_model.predict(features)
        log_comparison(
            prod=prod_result,
            shadow=shadow_result,
            diff=abs(prod_result - shadow_result)
        )

    asyncio.create_task(shadow_eval()) # إطلاق المهمة بنظام Fire & Forget
    return {"duration_min": round(float(prod_result), 2)}
```

عن أبي هريرة رضي الله عنه قال: قال النبي صلى الله عليه وعلى آله وسلم : ((يقول الله تعالى: أنا عند ظن عبدي بي وأنا معه إذا ذكرني فإن ذكرني في نفسه ذكرته في نفسي ، وإن ذكرني في ملأٍ ذكرته في ملأٍ خيرٍ منهم، وإن تقرب إلي شبرا تقربت إليه ذراعاً ، وإن تقرب إلي ذراعاً تقربت إليه باعاً ، وإن أتاني يمشي أتيته هرولة)). (حديث صحيح)

سبحان الله العظيم

سبحان الله وبحمده

استغفر الله العظيم

لا حول ولا قوة الا بالله

اللهم صل على محمد